

Teoria della dimostrazione, Teoria dei tipi, Proof Assistants e AI

verso una nuova pratica della matematica

Marino Miculan

Scuola Estiva di Logica AILA 2026

Bardonecchia, 31 agosto – 5 settembre 2026

DMIF, Università di Udine | marino.miculan@uniud.it | <https://marino.miculan.org>

LEZIONE 4

Laboratorio Lean I: logica e induzione al calcolatore

AILA 2026 — Bardonecchia

Obiettivo

Far **toccare con mano** esattamente ciò che è stato dimostrato astrattamente in L1 e L3.

Su che cosa poggia Lean

Il nucleo è **CIC**  L3: Prop *impredicativo*, *induttivi*, *universi*, con **proof irrelevance** **definizionale**. Sopra, *tre* assiomi — e sono tutti.

I TRE ASSIOMI — `#print axioms` li elenca

`propext` $(a \leftrightarrow b) \rightarrow a = b$

`Quot.sound` $\forall r : \alpha \rightarrow \alpha \rightarrow \text{Prop}, \forall a b : \alpha, r a b \rightarrow [a]_r = [b]_r$

`Classical.choice` $\text{Nonempty } \alpha \rightarrow \alpha$

Che cosa *non* è un assioma

La proof irrelevance non sta in questa lista: non è un assioma, è una **regola del kernel**. Due prove della stessa proposizione sono uguali *per definizione*, senza che nessuno lo debba assumere.

Che cosa ne segue: siamo in HOL classica

Da quei tre assiomi, più la proof irrelevance, discendono **come teoremi** tutti gli assunti della matematica classica.

Quot.sound	$\Rightarrow$	funext
proof irrelevance	$\Rightarrow$	UIP / K : $\forall h_1 h_2 : a = b, h_1 = h_2$, per rfl
scelta + propext + funext	$\Rightarrow$	Classical.em  : Diaconescu

In MLTT K non è derivabile  L3: Hofmann–Streicher; qui è gratis. Per questo Lean e HoTT non possono stare insieme.

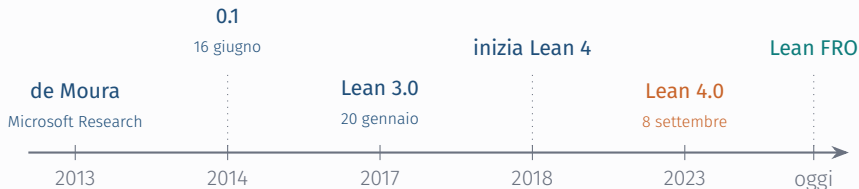
Dove ci troviamo, in pratica

Prop è booleana a due valori, le funzioni sono estensionali, c'è la scelta: sono *esattamente* gli assiomi della **logica classica di ordine superiore**. Sotto restano però tipi dipendenti, universi e induttivi — e `#print axioms` tiene la distinzione **ispezionabile**, cosa che in HOL non si può fare.

to combine the high level of trust provided by a small, independently-implementable logical kernel with the convenience and automation of tools like SMT solvers

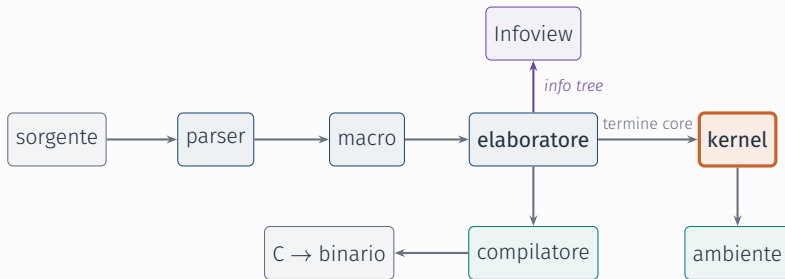
— Lean Language Reference

- ▶ lo stesso linguaggio in cui si dimostrano teoremi è quello in cui si scrivono programmi — e in cui è scritto Lean stesso: circa il **90%** dell'implementazione è in Lean;
- ▶ tattiche, notazioni e nuovi comandi si definiscono *dentro* il linguaggio 🔗 L1: la linea LCF, senza metalinguaggio separato;
- ▶ sotto, il nucleo è CIC 🔗 L3: Prop, induttivi, quozienti.



- ▶ **Lean 3** porta l'adozione di massa fra i matematici: Mathlib supera il milione di righe;
- ▶ **Lean 4** è riscritto in Lean. Nonostante Mathlib sia cresciuta del 50% durante il porting, la controlla *più in fretta* di quanto Lean 3 controllasse la versione più piccola;
- ▶ dal 2023 lo sviluppo è alla **Lean FRO**, una non-profit fondata da de Moura e Ullrich.

Che cosa succede quando compili un .lean



⚠ Dove sta la fiducia

L'elaboratore è enorme — unificazione, type class, inferenza, tattiche — e **non** è nella base fidata. Il kernel riceve un termine già esplicito: non unifica e non ha un controllo di terminazione, perché la ricorsione è già stata tradotta in usi dei ricursori [↻ L3: Nat.rec](#).

Lean non compila il file *tutto insieme*: legge un comando, lo elabora, lo fa controllare al kernel, aggiorna l'ambiente, e passa al successivo.

- ▶ per questo una **notation** o una **macro** è utilizzabile *dalla riga dopo*: il parser stesso è stato esteso;
- ▶ per questo l'editor può dirti che cosa sa *in quel punto* del file: gli *info tree* prodotti dall'elaboratore sono ciò che alimenta l'Infoview;
- ▶ per questo un errore a metà file non impedisce di lavorare sul resto.

La compilazione vera e propria è un ramo a parte: un file C per modulo, poi codice nativo. Serve a *eseguire*, non a *fidarsi*.

Notazione: come si scrivono i simboli

Si digita la **barra rovesciata** seguita dal nome, poi spazio o tab. Passando il mouse su un simbolo, VS Code mostra l'abbreviazione che lo produce.

$\rightarrow$	<code>\to \r</code>	$\forall$	<code>\forall</code>	<code>\forall</code>	<code>\forall</code>	$\langle \rangle$	<code>\< \></code>
λ	<code>\lambda \l</code>	$\exists$	<code>\exists</code>	<code>\exists</code>	<code>\exists</code>	$\cdot$	<code>\cdot</code>
$\wedge$	<code>\and</code>	$\neg$	<code>\not</code>	<code>\not</code>	<code>\not</code>	$^{-1}$	<code>\inv</code>
$\vee$	<code>\or</code>	$\neq$	<code>\ne</code>	<code>\ne</code>	<code>\ne</code>	$\mathbb{N} \mathbb{R}$	<code>\N \R</code>
$\leftrightarrow$	<code>\iff</code>	$\leq$	<code>\le</code>	<code>\le</code>	<code>\le</code>	$\alpha \beta$	<code>\a \b</code>

⚠ Attenzione

Alcuni simboli hanno un'alternativa ASCII ($\rightarrow$ per $\rightarrow$, **fun** per λ , $\leq$ per $\leq$), la maggior parte no. E attenzione: **!= non** è $\neq$ — è **bne**, la disuguaglianza booleana.

Dove cercare

PER IMPARARE

- ▶ *Theorem Proving in Lean 4* — il manuale, dalle basi;
- ▶ Avigad e Massot, *Mathematics in Lean* — matematica vera, con esercizi;
- ▶ *Natural Number Game*
<https://adam.math.hhu.de>

PER ANDARE OLTRE MATHLIB

CSLib — componenti riusabili per l'informatica: modelli di calcolo, complessità.
<https://www.cslib.io>

Tutti raccolti anche nel **README.md** del repo, con il **CHEATSHEET.md**.

PER CERCARE UN LEMMA

- ▶ documentazione di Mathlib
https://leanprover-community.github.io/mathlib4_docs/
- ▶ **Loogle** — per *forma*
<https://loogle.lean-lang.org>
- ▶ **LeanSearch** — in inglese
<https://leansearch.net>

PER CHIEDERE

Zulip:

<https://leanprover.zulipchat.com>

Prima di cominciare: il setup

Opzione A — locale

- ▶ `elan` + VS Code + estensione Lean 4;
- ▶ progetto Mathlib **già compilato**;
- ▶ `lake exe cache get` scarica GB: **va fatto prima di arrivare**.

Opzione B — zero install

- ▶ GitHub Codespaces, dal repo: *Code* → *Codespaces*;
- ▶ <https://live.lean-lang.org> — ha Mathlib, ma è lento e non salva il lavoro.

Repository del corso

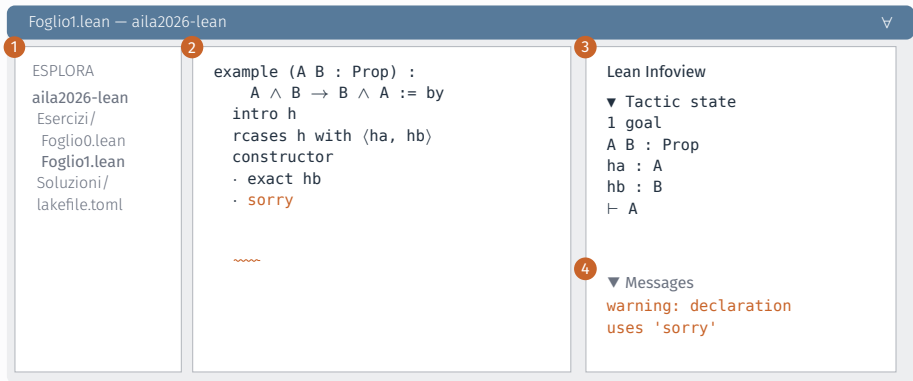
<https://github.com/miculan/aila2026-lean>

Cinque fogli in **Esercizi/**, con i **sorry** da riempire; le soluzioni commentate in **Soluzioni/**; un **CHEATSHEET.md** con tattiche ed errori comuni.

Blocco 1

Tour dell'ambiente — Foglio0

L'editor: che cosa si vede



1 apri la **cartella**, non il file

3 l'icona ∇ apre l'**Infoview**: il goal sotto il cursore

2 rosso = errore, giallo = **sorry**

4 output di **#check** e **#eval**

```
import Mathlib.Tactic

#check (2 + 2 : Nat)  -- 2 + 2 : Nat
#eval 2 + 2           -- 4

/- term mode: la dimostrazione È il termine -/
example (A : Prop) (h : A) : A := h

/- tactic mode: costruiamo il termine per passi -/
example (A : Prop) (h : A) : A := by
  exact h
```

- ▶ l'**Infview** mostra il *goal*: ipotesi sopra la barra, tesi sotto — è un contesto Γ e un tipo A ;
- ▶ **theorem**/**lemma** nominano il risultato, **example** no.

Come si legge un goal

```
A B C : Prop  
h1 : A → B  
h2 : B → C  
a : A
```

```
⊢ C
```



$\underbrace{A, B, C : \text{Prop}, h_1 : A \rightarrow B, h_2 : B \rightarrow C, a : A}_{\Gamma} \vdash ? : C$

La lista di ipotesi è il contesto Γ , la barra è il tornello, e il $?$ è il termine di prova che stiamo costruendo.  L1: lo stesso giudizio $\Gamma \vdash t : A$

Blocco 2

Logica proposizionale come esercizio Curry–Howard — Foglio 1

L'esempio canonico, di nuovo

 L1: lo stesso termine costruito alla lavagna

```
-- term mode: il  $\lambda$ -termine, identico a quello di L1
example (A B C : Prop) (h1 : A  $\rightarrow$  B) (h2 : B  $\rightarrow$  C) : A  $\rightarrow$  C :=
  fun a => h2 (h1 a)

-- tactic mode: gli stessi passi, dall'altro verso
example (A B C : Prop) (h1 : A  $\rightarrow$  B) (h2 : B  $\rightarrow$  C) : A  $\rightarrow$  C := by
  intro a      --  $\rightarrow$ -intro : sposta A nel contesto
  exact h2 (h1 a)
```

Intuizione

intro è $\rightarrow$ I. **exact**/**apply** sono $\rightarrow$ E. Le tattiche costruiscono il termine di prova: **#print** lo mostra.

tattica	regola
intro	$\rightarrow I, \Pi I$
exact	chiude il goal
apply	$\rightarrow E$ all'indietro
constructor	$\wedge I$
left/right	$\vee I_{1,2}$
cases/rcases	$\vee E, \wedge E$
exfalso	$\perp E$

```
example (A B : Prop) : A  $\wedge$  B  $\rightarrow$  B  $\wedge$  A
  := by
  intro h
  rcases h with ⟨ha, hb⟩
  constructor
  · exact hb
  · exact ha
```

```
example (A B : Prop) : A  $\vee$  B  $\rightarrow$  B  $\vee$  A
  := by
  rintro (ha | hb)
  · right; exact ha
  · left; exact hb
```

L'esperimento: $\neg\neg A \rightarrow A$

```
example (A : Prop) :  $\neg\neg A \rightarrow A$  := by
  intro h
  --  $\vdash A$  , con  $h : \neg\neg A$  . Provare: intro? exact? apply h?
  sorry
```

L'esperimento: $\neg\neg A \rightarrow A$

```
example (A : Prop) :  $\neg\neg A \rightarrow A$  := by
  intro h
  --  $\vdash A$  , con  $h : \neg\neg A$  . Provare: intro? exact? apply h?
  sorry
```

⚠ Non si chiude

Costruttivamente non c'è modo: BHK non fornisce alcun metodo per estrarre una dimostrazione di A da $(A \rightarrow \perp) \rightarrow \perp$. [🔗](#) L1: ecco che cosa significava, concretamente

L'esperimento: $\neg\neg A \rightarrow A$

```
example (A : Prop) :  $\neg\neg A \rightarrow A$  := by
  intro h
  --  $\vdash A$  , con  $h : \neg\neg A$  . Provare: intro? exact? apply h?
  sorry
```

⚠ Non si chiude

Costruttivamente non c'è modo: BHK non fornisce alcun metodo per estrarre una dimostrazione di A da $(A \rightarrow \perp) \rightarrow \perp$. [🔗](#) L1: ecco che cosa significava, concretamente

```
example (A : Prop) :  $\neg\neg A \rightarrow A$  := by
  intro h
  by_contra hA          -- Classical.byContradiction
  exact h hA
```

Che cosa abbiamo appena fatto, e a che prezzo

- ▶ **by_contra non** è una tattica come le altre: fa passare la prova per `Classical.em`
 : *stamattina: un teorema, non un assioma*;
- ▶ `#print axioms` lo rende visibile, ed è tutto il punto: la versione con **by_contra** elenca `Classical.choice`, l'altra no.

Che sia *lecito* lo sappiamo da ieri: il modello in ZFC  L3: *Carneiro, Werner*.

⚠ Il prezzo non è la coerenza: è la canonicità

Un termine chiuso di tipo $\mathbb{N}$ costruito con la scelta può non ridursi a un numerale: cadono disgiunzione ed esistenza, e da $\exists x. P(x)$ non si estrae più un testimone. In Lean quelle definizioni si marciano **noncomputable**.

🔧 Esercizio

Dimostrare $A \rightarrow \neg\neg A$ e $\neg\neg\neg A \rightarrow \neg A$, poi `#print axioms`: non useranno la scelta.

Blocco 3

Tipi induttivi e induzione — Foglio2

$\mathbb{N}$ come tipo induttivo

```
inductive MyNat where  
  | zero : MyNat  
  | succ : MyNat → MyNat  
  
#check @MyNat.rec    -- l'eliminatore: È il principio di induzione
```

🔗 L3: $\text{ind}_{\mathbb{N}}$, scritto alla lavagna: è letteralmente questo

```
def add : Nat → Nat → Nat  
  | n, 0      => n                -- ricorsione sul SECONDO argomento  
  | n, .succ m => .succ (add n m)
```

$n + 0$ contro $0 + n$: la differenza di L3

```
-- definizionale: il kernel calcola, non serve dimostrare nulla
theorem add_zero_nat (n : Nat) : n + 0 = n := rfl

-- proposizionale: serve induzione
theorem zero_add_nat (n : Nat) : 0 + n = n := by
  induction n with
    | zero      => rfl
    | succ n ih => simp [Nat.add_succ, ih]
```

💡 Perché l'asimmetria

$+$ è definito per ricorsione sul *secondo* argomento: quindi $n + 0$ si riduce a n per β/ι , cioè $n + 0 \equiv n$; mentre $0 + n$ è bloccato finché n è una variabile. [🔗 L3](#): $\equiv$ vs Id

Esercizio guidato: commutatività

```
theorem my_add_comm (m n : Nat) : m + n = n + m := by  
  induction n with  
    | zero      => simp  
    | succ n ih => rw [Nat.add_succ, ih, Nat.succ_add]
```

Somma di Gauss

```
theorem gauss (n : Nat) : 2 * ( $\sum i \in \text{Finset.range } (n+1), i$ ) = n * (n+1) := by  
  sorry
```

✂ In autonomia o in coppia

1. $(A \rightarrow B \rightarrow C) \rightarrow (A \wedge B \rightarrow C)$ e il viceversa (*currying*);
2. le leggi di De Morgan: quali valgono costruttivamente?
3. `length (l ++ l') = length l + length l'` su **List**;
4. `reverse (reverse l) = l`.

Nota

Sono nel **Foglio2.lean**, insieme agli altri. Le soluzioni stanno in **Soluzioni/** — da aprire *dopo* averci provato. Le rivediamo a inizio L5.

- ▶ J. Avigad e P. Massot. *Mathematics in Lean*. Documentazione online, aggiornata di continuo. 2025. URL: https://leanprover-community.github.io/mathematics_in_lean/ — capitoli 1–3.
- ▶ J. Avigad et al. *Theorem Proving in Lean 4*. 2025. URL: https://leanprover.github.io/theorem_proving_in_lean4/
- ▶ L. de Moura e S. Ullrich. «The Lean 4 Theorem Prover and Programming Language». In: *Automated Deduction (CADE-28)*. Vol. 12699. LNCS. Springer, 2021, pp. 625–635
- ▶ M. Carneiro. «The Type Theory of Lean». Tesi di laurea mag. Carnegie Mellon University, 2019 — il modello che giustifica gli assiomi.